



UNIVERSITÉ DU QUÉBEC À RIMOUSKI

DÉPARTEMENT DE MATHÉMATIQUES, D'INFORMATIQUE ET DE GÉNIE

Rapport du travail pratique n°3 :

Système de Visionnement de Films en Ligne

Dans le cadre du cours
Génie Logiciel II (INF23207-MS)
Professeur : **Ismail Khriss**

Réalisé par :

Mayeul Astride Samba
Simon Kasongo
Sony Sylvéus

Automne 2025

Table des matières

- 1. Introduction 3
- 2. Diagramme de Classes et Décisions de Conception 3
 - 2.1. Diagramme de Classes..... 3
 - 2.2. Description du Diagramme 7
 - 2.3. Décisions de Conception 16
- 3. Conclusion 24
- Références 25

1. Introduction

Ce rapport présente le diagramme de classes et les décisions de conception de notre système « **KASOMAFIX** », une application de visionnement de films en ligne développée dans le cadre du cours INF23207 - Génie logiciel II.

Le système a été conçu selon une **architecture Clean (en oignons)** et implémenté en utilisant **.NET 8.0** avec **C#**, **WPF** pour l'interface utilisateur, **Entity Framework Core** pour l'accès aux données et **SQL Server** comme système de gestion de base de données.

L'objectif de ce rapport est de documenter les choix de conception architecturale et de modélisation qui ont guidé le développement du système, en justifiant chaque décision par rapport aux exigences fonctionnelles et non fonctionnelles du projet.

2. Diagramme de Classes et Décisions de Conception

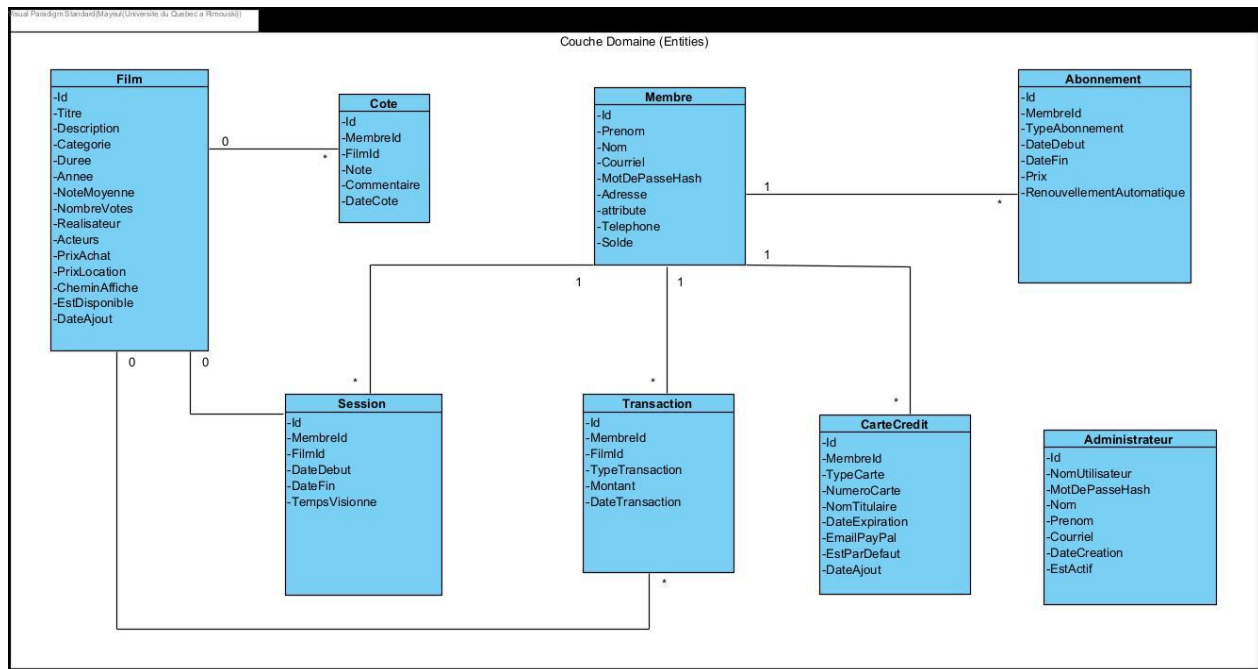
2.1. Diagramme de Classes

Le diagramme de classes représente la structure du modèle d'analyse paramétré selon nos décisions de conception. Il illustre les entités du domaine, leurs relations, les interfaces des repositories et les DTOs utilisés pour la communication entre les couches.

Le diagramme est organisé en trois couches principales correspondant à l'architecture Clean :

1. **Couche Domaine (Domain)** : Contient les 8 entités métier et leurs relations
2. **Couche Application (Application)** : Contient les 7 DTOs pour le transfert de données.
3. **Couche Infrastructure (Infrastructure)** : Contient les 8 interfaces de repositories

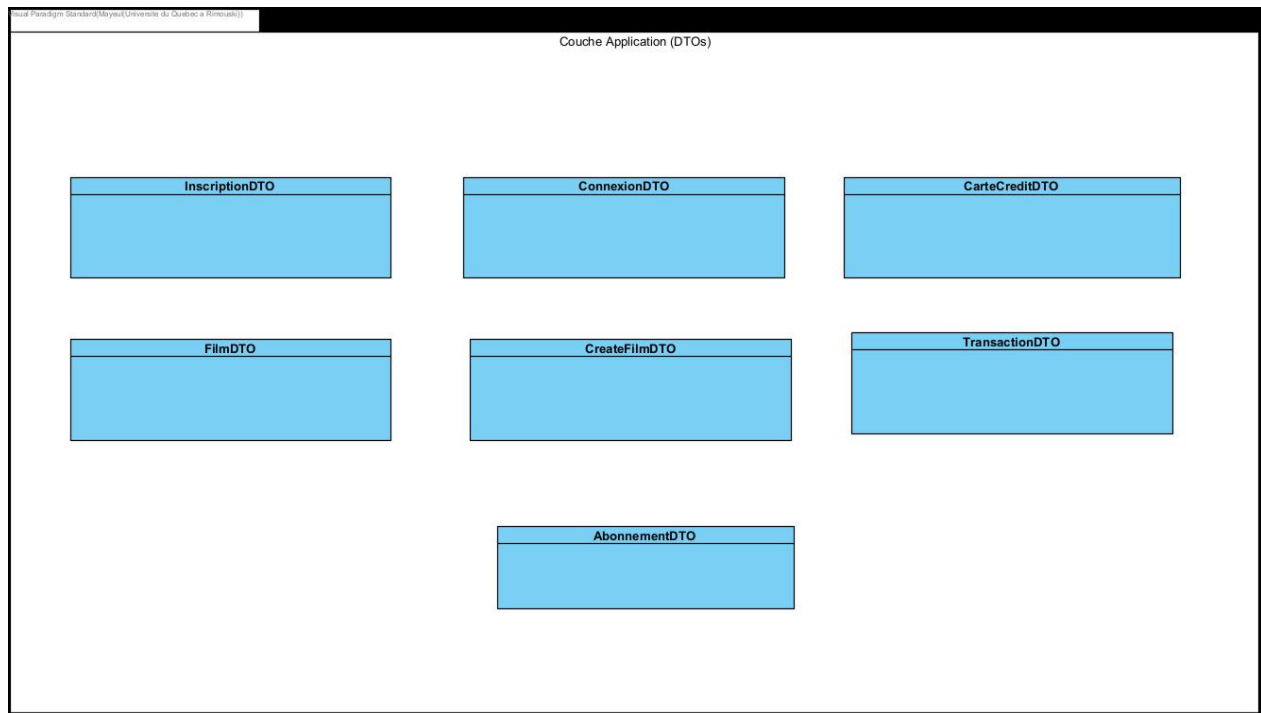
2.1.1. Diagramme de la Couche Domaine



Le diagramme de la couche Domaine présente les 8 entités principales du système :

- **Membre** : Représente un utilisateur inscrit
- **Film** : Représente un film dans le catalogue
- **Abonnement** : Représente un abonnement d'un membre
- **Transaction** : Représente une transaction financière
- **Session** : Représente une session de visionnement
- **Cote** : Représente une note et commentaire
- **CarteCredit** : Représente un mode de paiement
- **Administrateur** : Représente un administrateur du système

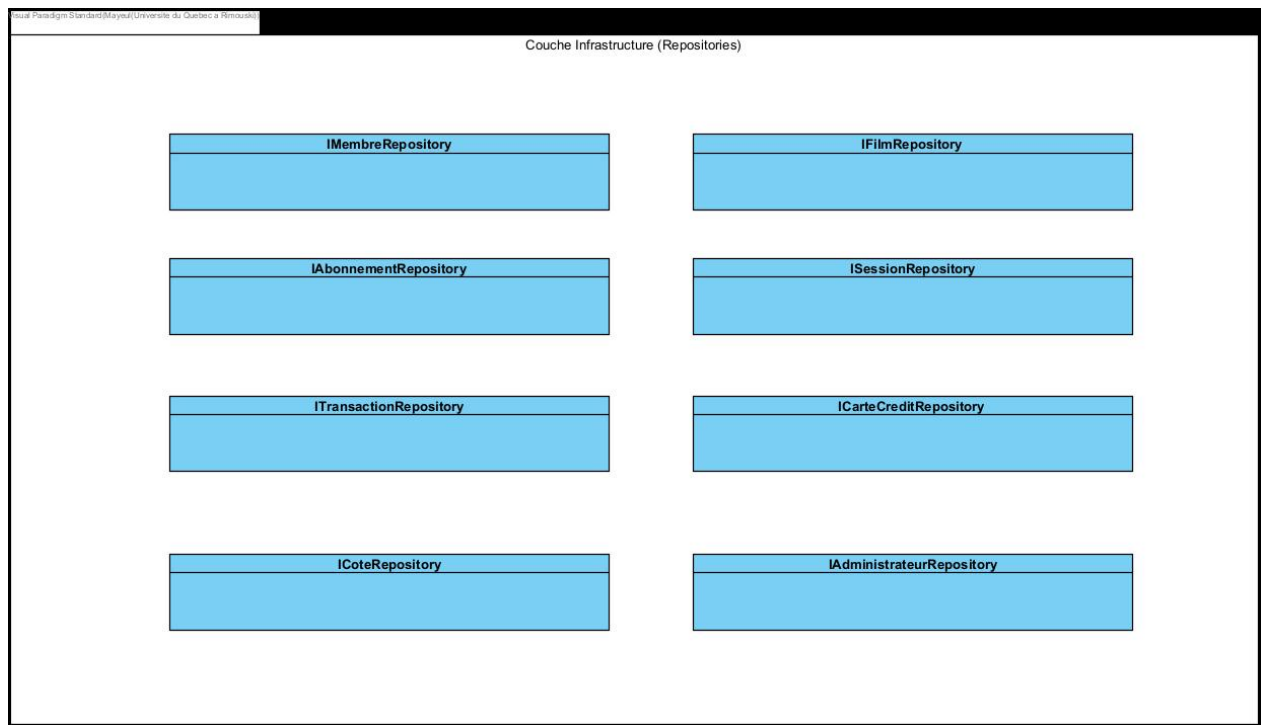
2.1.2. Diagramme de la Couche Application



Le diagramme de la couche Application présente les 7 DTOs principaux :

- **InscriptionDTO** : Pour l'inscription d'un nouveau membre
- **ConnexionDTO** : Pour la connexion (membre ou admin)
- **FilmDTO** : Pour les informations d'un film
- **CreateFilmDTO** : Pour créer ou modifier un film
- **TransactionDTO** : Pour l'historique des transactions
- **AbonnementDTO** : Pour les informations d'abonnement
- **CarteCreditDTO** : Pour les modes de paiement

2.1.3. Diagramme de la Couche Infrastructure



Le diagramme de la couche Infrastructure présente les 8 interfaces de repositories :

- **IMembreRepository** : Opérations CRUD pour les membres
- **IFilmRepository** : Opérations CRUD et recherche pour les films
- **IAbonnementRepository** : Gestion des abonnements
- **ISessionRepository** : Gestion des sessions de visionnement
- **ITransactionRepository** : Gestion des transactions
- **ICarteCreditRepository** : Gestion des modes de paiement
- **ICoteRepository** : Gestion des notes et commentaires
- **IAdministrateurRepository** : Gestion des administrateurs

2.2. Description du Diagramme

2.2.1. Entités du Domaine

Membre

L'entité **Membre** représente un utilisateur inscrit du système. Elle contient les informations personnelles et le solde du compte.

Propriétés :

- Id : Identifiant unique (int)
- Prenom : Prénom du membre (string, max 100)
- Nom : Nom du membre (string, max 100)
- Courriel : Adresse courriel unique (string, max 255, index unique)
- MotDePasseHash : Hash du mot de passe (string, max 255)
- Adresse : Adresse postale (string, max 500) - Telephone : Numéro de téléphone (string, max 20)
- Solde : Solde du compte en dollars (decimal(18,2), défaut 0)
- DateInscription : Date d'inscription (DateTime)

Relations :

- **1..*** avec Abonnement : Un membre peut avoir plusieurs abonnements (cardinalité 1 côté Membre, * côté Abonnement)
- **1..*** avec Transaction : Un membre peut avoir plusieurs transactions (cardinalité 1 côté Membre, * côté Transaction)
- **1..*** avec Session : Un membre peut avoir plusieurs sessions de visionnement (cardinalité 1 côté Membre, * côté Session)
- **1..*** avec Cote : Un membre peut noter plusieurs films (cardinalité 1 côté Membre, * côté Cote)
- **1..*** avec CarteCredit : Un membre peut avoir plusieurs modes de paiement (cardinalité 1 côté Membre, * côté CarteCredit)

Film

L'entité **Film** représente un film dans le catalogue. Elle contient toutes les informations nécessaires pour l'affichage et la gestion.

Propriétés :

- Id : Identifiant unique (int)
- Titre : Titre du film (string, max 200, requis)
- Description : Synopsis du film (string, max 2000)
- Categorie : Catégorie du film (string, max 100, requis)
- Duree : Durée en minutes (int)
- Annee : Année de sortie (int)
- NoteMoyenne : Note moyenne calculée (decimal(3,2), défaut 0)
- NombreVotes : Nombre de votes reçus (int, défaut 0)

- Realisateur : Nom du réalisateur (string, max 200) - Acteurs : Liste des acteurs séparés par virgules (string, max 500)
- PrixAchat : Prix d'achat (decimal(10,2))
- PrixLocation : Prix de location (decimal(10,2))
- CheminAffiche : Chemin vers l'image de l'affiche (string, max 500)
- EstDisponible : Indique si le film est disponible (bool, défaut true)
- DateAjout : Date d'ajout au catalogue (DateTime)

Relations :

- **0..*** avec Transaction : Un film peut être associé à plusieurs transactions (cardinalité 0 côté Film, * côté Transaction). La relation est optionnelle car certaines transactions (abonnements, ajout de solde) ne sont pas liées à un film.
- **0..*** avec Session : Un film peut être visionné dans plusieurs sessions (cardinalité 0 côté Film, * côté Session)
- **0..*** avec Cote : Un film peut recevoir plusieurs notes (cardinalité 0 côté Film, * côté Cote)

Abonnement

L'entité **Abonnement** représente un abonnement mensuel ou annuel d'un membre.

Propriétés :

- Id : Identifiant unique (int)
- Membrelid : Clé étrangère vers Membre (int, requis)
- TypeAbonnement : Type d'abonnement (string, ex: "Mensuel", "Annuel")
- DateDebut : Date de début (DateTime)
- DateFin : Date de fin (DateTime)
- Prix : Prix de l'abonnement (decimal(10,2))
- RenouvellementAutomatique : Indique si le renouvellement est automatique (bool, défaut true)
- EstActif : Indique si l'abonnement est actif (bool, défaut true)

Relations :

- **1** avec Membre : Chaque abonnement appartient à un membre (cardinalité 1 côté Abonnement, 1 côté Membre, FK: Membrelid, suppression en cascade)

Transaction

L'entité **Transaction** représente une transaction financière (achat, location, abonnement, ajout de solde).

Propriétés :

- Id : Identifiant unique (int)
- Membrelid : Clé étrangère vers Membre (int, requis)

- FilmId : Clé étrangère vers Film (int?, nullable)
- TypeTransaction : Type de transaction (string, ex: "Achat", "Location", "Abonnement", "AjoutSolde")
- Montant : Montant de la transaction (decimal(10,2))
- DateTransaction : Date de la transaction (DateTime, défaut maintenant)
- Statut : Statut de la transaction (string, défaut "Complétée")

Relations :

- **1** avec Membre : Chaque transaction appartient à un membre (cardinalité 1 côté Transaction, 1 côté Membre, FK: Membred, suppression restreinte)
- **0..1** avec Film : Une transaction peut être liée à un film (cardinalité 1 côté Transaction, 0..1 côté Film, FK: FilmId, nullable, suppression restreinte)
- **1** avec CarteCredit : Chaque transaction peut être associée à une carte de crédit (cardinalité 1 côté Transaction, * côté CarteCredit). Cette relation permet de tracer quel mode de paiement a été utilisé pour chaque transaction.

Note : FilmId est nullable car certaines transactions (abonnement, ajout de solde) ne sont pas liées à un film spécifique.

Session

L'entité **Session** représente une session de visionnement d'un film par un membre.

Propriétés :

- Id : Identifiant unique (int)
- Membred : Clé étrangère vers Membre (int, requis)
- FilmId : Clé étrangère vers Film (int, requis) - DateDebut : Date et heure de début (DateTime)
- DateFin : Date et heure de fin (DateTime?, nullable)
- TempsVisionne : Temps de visionnement en secondes (int)
- EstTerminee : Indique si la session est terminée (bool, défaut false)

Relations :

- **1** avec Membre : Chaque session appartient à un membre (cardinalité 1 côté Session, 1 côté Membre, FK: Membred, suppression en cascade)
- **1** avec Film : Chaque session concerne un film (cardinalité 1 côté Session, 0 côté Film, FK: FilmId, suppression restreinte)

Cote

L'entité **Cote** représente une note et un commentaire qu'un membre a donnés à un film.

Propriétés :

- Id : Identifiant unique (int)
- MembrelId : Clé étrangère vers Membre (int, requis)
- FilmId : Clé étrangère vers Film (int, requis)
- Note : Note de 1 à 5 (int)
- Commentaire : Commentaire optionnel (string?, nullable)
- DateCote : Date de la note (DateTime, défaut maintenant)

Relations :

- **1** avec Membre : Chaque note appartient à un membre (cardinalité * côté Cote, 1 côté Membre, FK: MembrelId, suppression en cascade)
- **1** avec Film : Chaque note concerne un film (cardinalité * côté Cote, 0 côté Film, FK: FilmId, suppression en cascade)

CarteCredit

L'entité **CarteCredit** représente un mode de paiement enregistré par un membre.

Propriétés :

- Id : Identifiant unique (int)
- MembrelId : Clé étrangère vers Membre (int, requis)
- TypeCarte : Type de carte (string, ex: "Visa", "Mastercard", "PayPal")
- NumeroCarte : Numéro de carte (seulement les 4 derniers chiffres stockés) (string, max 20) - NomTitulaire : Nom du titulaire (string, max 100)
- DateExpiration : Date d'expiration (DateTime)
- EmailPayPal : Email PayPal (si applicable) (string?, nullable, max 255)
- EstParDefaut : Indique si c'est le mode de paiement par défaut (bool, défaut false) - DateAjout : Date d'ajout (DateTime, défaut maintenant)

Relations :

- **1** avec Membre : Chaque carte appartient à un membre (cardinalité * côté CarteCredit, 1 côté Membre, FK: MembrelId, suppression en cascade)
- **0..*** avec Transaction : Une carte de crédit peut être utilisée pour plusieurs transactions (cardinalité * côté CarteCredit, 1 côté Transaction). Cette relation permet de tracer l'historique des paiements par mode de paiement.

Administrateur

L'entité **Administrateur** représente un administrateur du système.

Propriétés :

- Id : Identifiant unique (int)
- NomUtilisateur : Nom d'utilisateur unique (string, max 50, index unique)
- MotDePasseHash : Hash du mot de passe (string, max 255)

- Nom : Nom de l'administrateur (string, max 100)
- Prenom : Prénom de l'administrateur (string, max 100)
- Courriel : Adresse courriel unique (string, max 255, index unique)
- DateCreation : Date de création du compte (DateTime, défaut maintenant)
- EstActif : Indique si le compte est actif (bool, défaut true)

Relations : Aucune (entité indépendante)

2.2.2. DTOs (Data Transfer Objects)

Les DTOs sont utilisés pour transférer les données entre les couches sans exposer les entités du domaine. Ils sont organisés dans la couche Application.

InscriptionDTO

DTO utilisé pour l'inscription d'un nouveau membre.

Propriétés :

- Prenom : Prénom du membre
- Nom : Nom du membre - Courriel : Adresse courriel
- MotDePasse : Mot de passe (en clair, sera hashé)
- Adresse : Adresse postale
- Telephone : Numéro de téléphone

ConnexionDTO

DTO utilisé pour la connexion d'un utilisateur (membre ou administrateur).

Propriétés :

- Identifiant : Nom d'utilisateur (admin) ou courriel (membre)
- MotDePasse : Mot de passe

FilmDTO

DTO utilisé pour afficher les informations d'un film.

Propriétés :

- Id : Identifiant du film
- Titre : Titre du film
- Description : Synopsis
- Categorie : Catégorie
- Duree : Durée en minutes
- Annee : Année de sortie
- NoteMoyenne : Note moyenne
- NombreVotes : Nombre de votes

- Realisateur : Nom du réalisateur
- Acteurs : Liste des acteurs
- PrixAchat : Prix d'achat
- PrixLocation : Prix de location
- CheminAffiche : Chemin vers l'affiche

CreateFilmDTO

DTO utilisé pour créer ou modifier un film.

Propriétés :

- Titre : Titre du film
- Description : Synopsis
- Categorie : Catégorie
- Duree : Durée en minutes
- Annee : Année de sortie - Realisateur : Nom du réalisateur
- Acteurs : Liste des acteurs
- PrixAchat : Prix d'achat
- PrixLocation : Prix de location
- CheminAffiche : Chemin vers l'affiche

TransactionDTO

DTO utilisé pour afficher l'historique des transactions.

Propriétés :

- Id : Identifiant de la transaction
- TypeTransaction : Type de transaction
- Montant : Montant - DateTransaction : Date de la transaction
- FilmTitre : Titre du film (si applicable)

AbonnementDTO

DTO utilisé pour afficher les informations d'un abonnement.

Propriétés :

- Id : Identifiant de l'abonnement
- TypeAbonnement : Type d'abonnement
- DateDebut : Date de début
- DateFin : Date de fin
- Prix : Prix de l'abonnement
- EstActif : Indique si l'abonnement est actif

CarteCreditDTO

DTO utilisé pour afficher les informations d'un mode de paiement.

Propriétés :

- Id : Identifiant de la carte
- TypeCarte : Type de carte
- NumeroCarte : Numéro de carte (4 derniers chiffres)
- NomTitulaire : Nom du titulaire - DateExpiration : Date d'expiration
- EstParDefaut : Indique si c'est le mode par défaut

2.2.3. Interfaces des Repositories

Les interfaces des repositories sont définies dans la couche Domain et implémentées dans la couche Infrastructure. Elles permettent l'abstraction de l'accès aux données.

IMembreRepository

Interface pour les opérations CRUD sur les membres.

Méthodes principales :

- GetByIdAsync(int id) : Récupère un membre par son ID
- GetByCourrielAsync(string courriel) : Récupère un membre par son courriel
- GetAllAsync() : Récupère tous les membres
- AddAsync(Membre membre) : Ajoute un nouveau membre
- UpdateAsync(Membre membre) : Met à jour un membre
- DeleteAsync(int id) : Supprime un membre
- ExistsByCourrielAsync(string courriel) : Vérifie si un courriel existe

IFilmRepository

Interface pour les opérations CRUD et recherche sur les films.

Méthodes principales :

- GetByIdAsync(int id) : Récupère un film par son ID
- GetAllAsync() : Récupère tous les films
- RechercherAsync(string? titre, string? categorie, int? annee) : Recherche des films selon des critères
- AddAsync(Film film) : Ajoute un nouveau film
- UpdateAsync(Film film) : Met à jour un film
- DeleteAsync(int id) : Supprime un film

IAbonnementRepository

Interface pour la gestion des abonnements.

Méthodes principales :

- GetByIdAsync(int id) : Récupère un abonnement par son ID
- GetByMembreIdAsync(int membreId) : Récupère les abonnements d'un membre
- GetActifsByMembreIdAsync(int membreId) : Récupère les abonnements actifs d'un membre
- AddAsync(Abonnement abonnement) : Ajoute un nouvel abonnement
- UpdateAsync(Abonnement abonnement) : Met à jour un abonnement

ISessionRepository

Interface pour la gestion des sessions de visionnement.

Méthodes principales :

- GetByIdAsync(int id) : Récupère une session par son ID
- GetByMembreIdAsync(int membreId) : Récupère les sessions d'un membre
- GetByFilmIdAsync(int filmId) : Récupère les sessions d'un film
- AddAsync(Session session) : Ajoute une nouvelle session
- UpdateAsync(Session session) : Met à jour une session

ITransactionRepository

Interface pour la gestion des transactions.

Méthodes principales :

- GetByIdAsync(int id) : Récupère une transaction par son ID
- GetByMembreIdAsync(int membreId) : Récupère les transactions d'un membre
- AddAsync(Transaction transaction) : Ajoute une nouvelle transaction
- GetRevenusAsync(DateTime? dateDebut, DateTime? dateFin) : Calcule les revenus sur une période

ICarteCreditRepository

Interface pour la gestion des modes de paiement.

Méthodes principales :

- GetByIdAsync(int id) : Récupère une carte par son ID
- GetByMembreIdAsync(int membreId) : Récupère les cartes d'un membre
- GetParDefautByMembreIdAsync(int membreId) : Récupère la carte par défaut d'un membre
- AddAsync(CarteCredit carte) : Ajoute une nouvelle carte
- UpdateAsync(CarteCredit carte) : Met à jour une carte
- DeleteAsync(int id) : Supprime une carte

ICoteRepository

Interface pour la gestion des notes et commentaires.

Méthodes principales :

- GetByIdAsync(int id) : Récupère une note par son ID
- GetByFilmIdAsync(int filmId) : Récupère les notes d'un film
- GetByMembreIdAsync(int membreId) : Récupère les notes d'un membre
- GetByMembreEtFilmAsync(int membreId, int filmId) : Récupère la note d'un membre pour un film - AddAsync(Cote cote) : Ajoute une nouvelle note
- UpdateAsync(Cote cote) : Met à jour une note

IAdministrateurRepository

Interface pour la gestion des administrateurs.

Méthodes principales :

- GetByIdAsync(int id) : Récupère un administrateur par son ID
- GetByNomUtilisateurAsync(string nomUtilisateur) : Récupère un administrateur par son nom d'utilisateur
- GetAllAsync() : Récupère tous les administrateurs
- AddAsync(Administrateur admin) : Ajoute un nouvel administrateur
- UpdateAsync(Administrateur admin) : Met à jour un administrateur

2.3. Décisions de Conception

Cette section présente les principales décisions de conception prises lors du développement du système, avec leurs justifications détaillées.

2.3.1. Architecture en Oignons (Clean Architecture)

Décision : Utilisation de l'architecture en oignons pour séparer les préoccupations et assurer la maintenabilité.

Justification :

L'architecture en oignons (Clean Architecture) a été choisie pour plusieurs raisons :

1. **Séparation des responsabilités** : Chaque couche a une responsabilité claire et bien définie. Le domaine (cœur) ne dépend d'aucune technologie externe, ce qui garantit l'indépendance de la logique métier.
2. **Indépendance du domaine** : Le domaine ne dépend d'aucune technologie externe (base de données, framework UI, etc.). Cela permet de changer l'implémentation technique sans affecter la logique métier.
3. **Testabilité** : La séparation des couches facilite les tests unitaires grâce à l'injection de dépendances. Chaque couche peut être testée indépendamment avec des mocks.
4. **Évolutivité** : L'architecture permet d'ajouter de nouvelles fonctionnalités sans modifier le code existant. Les nouvelles fonctionnalités s'ajoutent comme de nouvelles couches ou extensions.
5. **Réutilisabilité** : La logique métier peut être réutilisée dans d'autres contextes (API Web, application mobile, etc.) sans modification.

Structure :

Domain (Cœur)

↑

Application (Use Cases, DTOs)

↑

Infrastructure (Repositories, EF Core)

↑

Presentation (WPF, Vues)

2.3.2. Stockage du Solde dans l'Entité Membre

Décision : Stockage du solde directement dans l'entité Membre plutôt que de le calculer dynamiquement à partir des transactions.

Justification :

1. **Performance :** Évite de recalculer le solde à chaque consultation en effectuant une somme sur toutes les transactions. Le solde est mis à jour lors de chaque transaction, garantissant une lecture rapide.
2. **Simplicité :** Accès direct au solde sans jointures complexes ou agrégations. Simplifie le code et améliore la lisibilité.
3. **Cohérence :** Le solde est mis à jour de manière atomique lors de chaque transaction, garantissant la cohérence des données. Les transactions sont effectuées dans le cadre d'une transaction de base de données, assurant l'intégrité.

Alternative considérée : Calculer le solde dynamiquement à partir des transactions. Cette approche a été rejetée car elle serait trop coûteuse en performance pour les consultations fréquentes.

2.3.3. Dénormalisation Contrôlée : NoteMoyenne et NombreVotes

Décision : Stockage de NoteMoyenne et NombreVotes comme propriétés calculées dans l'entité Film plutôt que de les calculer dynamiquement.

Justification :

1. **Performance :** Évite de calculer la moyenne à chaque consultation du film. Le calcul est effectué uniquement lors de l'ajout ou de la modification d'une note.
2. **Dénormalisation contrôlée :** Bien que cela viole la troisième forme normale, cette dénormalisation est acceptable car :
 - Les données sont mises à jour de manière cohérente (lors de l'ajout/modification d'une note)
 - Les performances de lecture sont grandement améliorées
 - Le risque d'incohérence est minimal et peut être géré par la logique métier

Alternative considérée : Calculer la moyenne dynamiquement à partir des notes. Cette approche a été rejetée car elle serait trop coûteuse pour les films populaires avec de nombreuses notes.

2.3.4. FilmId Nullable dans Transaction

Décision : La propriété FilmId dans l'entité Transaction est nullable pour permettre les transactions sans film associé.

Justification :

1. **Flexibilité :** Permet différents types de transactions :
 - **Achat :** Transaction liée à un film
 - **Location :** Transaction liée à un film
 - **Abonnement :** Transaction non liée à un film
 - **AjoutSolde :** Transaction non liée à un film
2. **Modélisation réaliste :** Toutes les transactions ne sont pas nécessairement liées à un film. Les abonnements et les ajouts de solde sont des transactions financières qui n'impliquent pas de film spécifique.
3. **Simplicité :** Évite de créer des entités séparées pour différents types de transactions, tout en maintenant la flexibilité nécessaire.

Alternative considérée : Créer des entités séparées (TransactionFilm, TransactionAbonnement, etc.). Cette approche a été rejetée car elle compliquerait inutilement le modèle et la logique métier.

2.3.5. Relation Transaction - CarteCredit

Décision : Ajout d'une relation entre Transaction et CarteCredit pour tracer quel mode de paiement a été utilisé pour chaque transaction.

Justification :

1. **Traçabilité :** Permet de savoir exactement quel mode de paiement a été utilisé pour chaque transaction, ce qui est important pour la gestion financière et le support client.
2. **Historique complet :** Maintient un historique complet des paiements par mode de paiement, facilitant les analyses et les rapports.
3. **Conformité :** Aide à respecter les exigences de traçabilité des transactions financières.

Note : Cette relation est optionnelle (0..* côté CarteCredit, 1 côté Transaction) car certaines transactions peuvent ne pas utiliser de carte de crédit enregistrée (paiement en espèces, virement, etc.).

2.3.6. Stockage Sécurisé des Numéros de Carte

Décision : Stockage de seulement les 4 derniers chiffres du numéro de carte dans l'entité CarteCredit.

Justification :

1. **Sécurité :** Conformité aux standards de sécurité PCI DSS (Payment Card Industry Data Security Standard). Les numéros de carte complets ne doivent jamais être stockés en clair.
2. **Confidentialité :** Protection des données sensibles des utilisateurs. Même en cas de compromission de la base de données, les numéros de carte complets ne peuvent pas être récupérés.
3. **Utilité :** Les 4 derniers chiffres sont suffisants pour identifier la carte pour l'utilisateur (affichage dans l'interface) sans compromettre la sécurité.

Implémentation : Lors de l'ajout d'une carte, seuls les 4 derniers chiffres sont extraits et stockés. Le numéro complet n'est jamais stocké dans la base de données.

2.3.7. Utilisation de Collections Virtuelles pour les Relations

Décision : Utilisation de collections virtuelles (`ICollection<T>`) pour les relations one-to-many dans Entity Framework.

Justification :

1. **Lazy Loading :** Entity Framework charge les collections à la demande grâce au lazy loading, évitant de charger des données inutiles.
2. **Performance :** Évite de charger toutes les données liées inutilement. Les collections sont chargées uniquement lorsqu'elles sont accédées.
3. **Flexibilité :** Permet d'ajouter/supprimer des éléments facilement grâce aux méthodes de collection.
4. **Eager Loading :** Permet également le chargement eager avec `Include()` lorsque nécessaire pour optimiser les requêtes.

Exemple :

```
public virtual ICollection<Transaction> Transactions { get; set; }
```

2.3.8. Comportements de Suppression (Cascade vs Restrict)

Décision : Utilisation de différents comportements de suppression selon la nature de la relation.

Justification :

1. **Cascade Delete :** Utilisé pour les relations où la suppression du parent doit supprimer les enfants :
 - Membre → Abonnement : Si un membre est supprimé, ses abonnements sont supprimés
 - Membre → Session : Si un membre est supprimé, ses sessions sont supprimées
 - Membre → Cote : Si un membre est supprimé, ses notes sont supprimées
 - Membre → CarteCredit : Si un membre est supprimé, ses cartes sont supprimées
 - Film → Cote : Si un film est supprimé, ses notes sont supprimées
2. **Restrict Delete :** Utilisé pour les relations où la suppression du parent ne doit pas supprimer les enfants (pour préserver l'historique) :
 - Membre → Transaction : Les transactions doivent être conservées pour l'historique financier
 - Film → Transaction : Les transactions doivent être conservées pour l'historique
 - Film → Session : Les sessions doivent être conservées pour l'historique de visionnement

Cette approche garantit l'intégrité des données tout en préservant l'historique important.

2.3.9. Utilisation de DTOs pour la Communication entre Couches

Décision : Utilisation de DTOs (Data Transfer Objects) pour transférer les données entre les couches.

Justification :

1. **Séparation des couches :** La présentation n'a pas accès direct aux entités du domaine. Cela garantit que la logique métier reste dans la couche Application.
2. **Sécurité :** Masque les données sensibles (ex: MotDePasseHash) qui ne doivent jamais être exposées à la présentation.
3. **Flexibilité :** Permet de transformer les données selon les besoins de l'interface sans modifier les entités du domaine.

4. **Performance** : Transfère uniquement les données nécessaires, évitant de charger des données inutiles.
5. **Évolutivité** : Permet de modifier la structure des entités sans affecter la présentation, et vice versa.

Exemples :

- FilmDTO : Expose uniquement les informations nécessaires à l’affichage (pas de DateAjout, EstDisponible si non nécessaire)
- CreateFilmDTO : Contient uniquement les données nécessaires à la création (pas d’Id, NoteMoyenne, etc.)
- TransactionDTO : Inclut le titre du film pour l’affichage sans charger l’entité complète

2.3.10. Pattern Repository

Décision : Utilisation du pattern Repository pour l’accès aux données.

Justification :

1. **Abstraction** : Cache les détails d’implémentation de l’accès aux données. La logique métier n’a pas besoin de savoir comment les données sont stockées.
2. **Testabilité** : Permet de créer des mocks pour les tests unitaires. Les Use Cases peuvent être testés sans accès à la base de données.
3. **Flexibilité** : Facilite le changement de source de données (SQL Server, MySQL, MongoDB, etc.) sans modifier la logique métier.
4. **Séparation des responsabilités** : La logique métier ne dépend pas de la base de données. Les Use Cases dépendent uniquement des interfaces, pas des implémentations.

Structure : - Interfaces définies dans la couche Domain - Implémentations dans la couche Infrastructure - Injection de dépendances pour l’utilisation dans les Use Cases

2.3.11. Implémentation de Use Cases Dédiés

Décision : Implémentation de chaque cas d’utilisation dans une classe dédiée.

Justification :

1. **Single Responsibility Principle** : Chaque UseCase a une responsabilité unique et bien définie. Facilite la compréhension et la maintenance.

2. **Maintenabilité** : Facilite la modification et le débogage. Chaque fonctionnalité est isolée dans sa propre classe.
3. **Testabilité** : Permet de tester chaque cas d'utilisation indépendamment. Les tests sont plus simples et plus ciblés.
4. **Documentation** : Le code devient auto-documenté. Le nom de la classe indique clairement ce qu'elle fait.

Exemple :

- InscriptionMembreUseCase : Gère uniquement l'inscription d'un nouveau membre
- ConnexionUseCase : Gère uniquement la connexion (membre ou admin)
- AjouterFilmUseCase : Gère uniquement l'ajout d'un film

2.3.12. Injection de Dépendances

Décision : Utilisation de l'injection de dépendances via Microsoft.Extensions.DependencyInjection.

Justification :

1. **Découplage** : Réduit les dépendances entre les composants. Les classes ne créent pas leurs dépendances, elles les reçoivent.
2. **Testabilité** : Facilite la création de tests avec des mocks. Les dépendances peuvent être remplacées facilement.
3. **Flexibilité** : Permet de changer l'implémentation sans modifier le code. Par exemple, changer de repository sans modifier les Use Cases.
4. **Gestion du cycle de vie** : Contrôle la création et la destruction des objets (Singleton, Scoped, Transient).

Configuration :

- Repositories : Scoped (une instance par requête)
- Use Cases : Scoped (une instance par requête)
- DbContext : Scoped (une instance par requête)

2.3.13. Hashage des Mots de Passe avec BCrypt

Décision : Utilisation de BCrypt pour le hashage des mots de passe.

Justification :

1. **Sécurité :** BCrypt est un algorithme de hashage sécurisé et résistant aux attaques. Il est conçu spécifiquement pour le hashage des mots de passe.
2. **Salt automatique :** BCrypt génère automatiquement un salt unique pour chaque mot de passe. Cela protège contre les attaques par rainbow tables.
3. **Résistance aux attaques :** Protection contre les attaques par force brute grâce au coût de calcul configurable.
4. **Standard de l'industrie :** BCrypt est largement utilisé et reconnu comme une solution sécurisée pour le hashage des mots de passe.

Implémentation : - Hashage lors de l'inscription ou de la modification du mot de passe -
Vérification lors de la connexion - Les mots de passe en clair ne sont jamais stockés

2.3.14 Entity Framework Core comme ORM

Décision : Utilisation d'Entity Framework Core comme ORM (Object-Relational Mapping).

Justification :

1. **Productivité :** Réduit le code boilerplate pour l'accès aux données. Pas besoin d'écrire du SQL manuellement pour les opérations CRUD simples.
2. **Type Safety :** Vérification des types à la compilation. Les erreurs sont détectées avant l'exécution.
3. **Migrations :** Facilite la gestion des changements de schéma. Les migrations sont générées automatiquement.
4. **LINQ :** Permet d'écrire des requêtes de manière déclarative et type-safe. Le code est plus lisible et maintenable.
5. **Intégration :** S'intègre parfaitement avec .NET et les autres technologies Microsoft.

Configuration : - Utilisation de `IEntityTypeConfiguration<T>` pour la configuration des entités
- Configuration séparée pour chaque entité dans des classes dédiées - Relations configurées avec comportements de suppression appropriés

2.3.15. Configuration Séparée des Entités

Décision : Utilisation de classes de configuration séparées (IEntityTypeConfiguration<T>) pour chaque entité.

Justification :

1. **Organisation :** Code mieux organisé et plus lisible. Chaque entité a sa propre classe de configuration.
2. **Maintenabilité :** Facilite la modification de la configuration. Les changements sont isolés dans une seule classe.
3. **Réutilisabilité :** Configuration réutilisable dans différents contextes (tests, migrations, etc.).
4. **Séparation des préoccupations :** Sépare la configuration de la logique métier. Les entités restent simples et focales.

Structure :

- Une classe de configuration par entité
- Configuration appliquée automatiquement via ApplyConfigurationsFromAssembly()
- Configuration centralisée dans le DbContext

3. Conclusion

Ce rapport a présenté le diagramme de classes et les principales décisions de conception du système KASOMAFIX. Les choix architecturaux et de modélisation ont été guidés par les principes de l'architecture Clean, privilégiant la séparation des responsabilités, la testabilité, la maintenabilité et la sécurité.

Les décisions prises permettent au système d'être :

- **Maintenable :** Code organisé et bien structuré
- **Testable :** Architecture facilitant les tests unitaires
- **Évolutif :** Facilite l'ajout de nouvelles fonctionnalités
- **Sécurisé :** Protection des données sensibles et conformité aux standards
- **Performant :** Optimisations appropriées (dénormalisation contrôlée, index)

Le système respecte les exigences du TP3 et implémente tous les cas d'utilisation requis tout en maintenant une architecture propre et professionnelle.

Les diagrammes présentés illustrent clairement la structure du système à travers ses trois couches principales : Domain (entités métier), Application (DTOs) et Infrastructure (repositories), démontrant une séparation claire des préoccupations conforme aux meilleures pratiques de l'architecture Clean.

Références

- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Microsoft. (2024). *Entity Framework Core Documentation*. <https://learn.microsoft.com/en-us/ef/core/>
- Microsoft. (2024). *Dependency Injection in .NET*. <https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection>
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.